STOCK ORACLE

Development Changelog

A full history of every version, what broke, why, and how it was fixed

---

Overview

This document tracks every major version of Stock Oracle from the initial concept to the current working build. Each version explains what was attempted, what went wrong, and what the next version did to fix it. What started as a Python script in March 2026 eventually became a full React application with real-time data from multiple APIs and AI-powered analysis.

Version	Description	Cost/Analysis	Status
v0.1	Python ML script	N/A	Replaced
v0.2	Flash + yfinance backend	N/A	Replaced
v0.3	Google Colab notebook	N/A	Replaced
v0.4	React + Claude + web search	$0.30+	Too expensive

v0.5	Shorter prompt	$0.21	Still too high
v0.6	System prompt + search cap	$0.40+	Got worse
v.0.7	No web search	0.01$	No live data
v0.8	CSS black screen fix	0.01$	Bugged fix
v0.9	Groq + yfinance	0.00$	Working
v1.0	CSS fix baked into App.jsx	0.00$	Working
v1.1	Finnhub + NewsAPI world events	0.00$	Working
v1.2	CORS fix + starts With crash fix	$0.00	Working
v1.3	Type safety fix - safeStr helper	$0.00	Working
v1.4	safeStr moved to top level scope	$0.00	Working
v1.5	Object rendering fix	$0.00	Working
v1.6	Prompt fix - Force real prices	$0.00	Working
v1.7	JSON repair - malformed response fix	$0.00	Working
v1.8	UI polish - ticker spacing + prediction detail	$0.00	Working
v1.9	Scan history + detailed analysis	$0.00	Working
v2.0	Full rebuild + clean file structure	$0.00	Working
v2.1	All features	$0.00	Working

	confirmed working end-to-end		
v2.2	Chart fix + 10 news items	$0.00	Working
v2.3	Twelve Data - real live prices	$0.00	Working
v2.4	News limit fix - 5 items only	$0.00	Working
v2.5	Dynamic search query for any ticker	$0.00	Working
v2.6	StockTwits sentiment panel	$0.00	FAILED (never worked)
v2.7	SEC EDGAR filings panel	$0.00	FAILED (never worked)
v2.8	Three working alternatives	$0.00	WORKING
v2.9	Real data replaces AI-generated numbers + probability ranges and conviction scoring	$0.00	WORKING
v3.0	Node.js backend + SEC EDGAR via proxy + full Finnhub integration	$0.00	Kinda Working
v3.1	Push to github	$0.00	Current

---

v0.1 - Python Machine Learning Script

Date: March 2026

Status: REPLACED

What it was

The very first version was a Python machine learning script I called ai_trading_model.py (later renamed to multi_stock_trading.py). It used scikit-learn and yfinance to download historical stock data, train a basic Random Forest model, and output buy/sell predictions right in the terminal. I was proud of it at the time, it felt like "real" AI.

Why it didn't work

Looking back, this thing had so many problems I don't even know where to start:

- You needed Python installed, plus scikit-learn, yfinance, pandas, numpy - it was a nightmare for anyone who wasn't already a developer.
- The ML model only looked at historical price patterns. It had no idea what was happening in the news, no concept of earnings reports, no understanding of market sentiment.
- There was no user interface at all, just raw terminal output with numbers.
- Markets don't care about pure historical patterns. My model would predict "BUY" based on some technical pattern while the company was getting hit with a massive lawsuit. Completely useless.

What I learned

I realized that if I wanted this to actually be useful, I needed two things: a proper user interface that normal people could use, and something much smarter than a basic ML

model. That's when I started thinking about using a real LLM that could understand context, news, and fundamentals - not just crunch numbers.

---

v0.2 - Flask Web App + yfinance Backend

Date: March 2026

Status: REPLACED

What it was

My next attempt was a Python Flask web server I called stock_oracle_app.py. It served an HTML frontend and the backend fetched real stock data using yfinance. You could actually open it in a browser and see stock prices. Progress.

Why it didn't work

But honestly, this still wasn't it:

- You still needed Python and Flask installed, and you had to run a local server every time you wanted to use it. Not exactly user-friendly.
- There was still no real AI analysis, just raw data from yfinance displayed on a webpage.
- Setup was way too complicated for anyone who wasn't technical.
- No predictions, no buy/sell signals, no recommendations. Just numbers on a screen.

What I learned

I realized the backend server model was too heavy. What I really needed was a fully client-side app that runs entirely in the browser. No Python, no Flask, no local server beyond a simple dev server for development. That's when I started thinking about React.

---

v0.3 - Google Colab Notebook

Date: March 2026

Status: REPLACED

What it was

I tried to make things easier by putting everything in a Google Colab notebook (stock_oracle_COLAB.py). The idea was that users could just open a link in their browser and run it without installing anything locally.

Why it didn't work

In theory this should have worked, but in practice:

- Colab notebooks are confusing for non-developers. Too many cells to run, too much setup, too easy to mess up.
- Still no real AI reasoning, just yfinance data with some basic calculations thrown in.
- The interface was still just raw Colab output. Not exactly what I would call a polished user experience.

What I learned

I finally accepted that this needed to be a proper web application with a real interface. No more notebooks, no more terminal scripts. I needed to learn React.

---

v0.4 - First React App (Claude API + Web Search)

Date: March 2026

Status: BROKEN (initially), then WORKING (after fixes)

What it was

My first proper React app (stock_oracle_web.jsx). This was exciting, it called the Anthropic Claude API directly from the browser and used the web_search tool so Claude could pull live data before generating analysis. It had a dark UI with a signal ring, price history chart, predictions, and news cards. It actually looked like a real application.

Why it didn't work at first

Of course, nothing works on the first try:

- The app called Claude's API directly from the browser, which is normally blocked by CORS. Browsers don't let you call external APIs without special headers.
- Fix: I found a workaround, added the anthropic-dangerous-direct-browser-access: true header to allow direct browser access. The name alone should have been a warning sign.
- The app was also rendering too small and cut off. Fix: Removed overflow:hidden and set minWidth to 100vw.

The cost problem - v0.4 cost $0.30+ per analysis

This is where things got really painful. Every analysis was costing over 30 cents. Why:

- The prompt included a massive JSON template example (about 400 tokens of example data) in every single request.
- The web_search tool was doing 5 to 10 searches per analysis and reading full web pages each time, thousands of tokens of input.
- max_tokens was set to 4096, which billed for the maximum possible output even when it was not needed.

At this rate, if someone did 10 analyses a day, that is $3. Over a month, almost $100.

Completely unsustainable.

---

v0.5 - Shortened Prompt (Still Claude API)

Date: March 2026

Status: PARTIAL FIX

What I changed

I replaced the long prompt with a compact one-line version. Removed the JSON template example. Dropped max_tokens to 1500.

Result

Cost dropped from $0.30 to about $0.21 per analysis. Better, but still way too high.

The web_search tool was still the main problem. Even with a short prompt, each search loaded full web pages which added thousands of extra input tokens that I could not control.

What I learned

The prompt length was only a small part of the cost. The real issue was the web search tool itself. Every time it ran, it ingested entire news articles and web pages, which added massive token counts no matter how short my prompt was.

---

v0.6 - System Prompt + Search Limit Attempt

Date: March 2026

Status: PARTIAL FIX (actually got worse)

What I changed

I moved the instructions from the user prompt into a system prompt, which Anthropic said could be cached and cost less to reuse. I also added max_uses: 2 to the web_search tool to try to cap it at 2 searches per analysis. Restored max_tokens to 4096 for full output quality.

Result

Cost went UP to $0.40+ per analysis. Somehow it got worse.

Why it failed

Turns out the max_uses parameter on the web search tool is not actually a supported field. Claude just ignored it and kept searching freely. And the system prompt caching

did not help enough because the web search results were still flooding the context with thousands of tokens.

---

v0.7 - Removed Web Search Entirely

Date: March 2026

Status: WORKING but USELESS

What I changed

I gave up and removed the web_search tool completely. Now Claude relied entirely on its training data. Cost dropped to about $0.01 to $0.02 per analysis. Finally, affordable.

The problem

But now there was a new problem: without live data, Claude was using knowledge that could be months old. Stock prices, recent news, earnings results, analyst upgrades - all outdated.

A stock analysis tool with no live data is essentially useless for making real decisions. You cannot tell someone to buy a stock at $150 when it is actually trading at $180 today.

What I learned

Cheap but pointless. We needed live data, but had to find a way to get it without paying Claude to read entire web pages.

---

v0.8 - Black Screen Bug

Date: March 2026

Status: BUG FIXED

What happened

After switching back to a version without web search, something weird happened: the app would load, run the analysis, then show a completely black or grey screen instead of the results. No error messages, just nothing.

Root cause

After hours of debugging, I finally figured it out. The index.css file (the default Vite template CSS) had this:

```
body {
display: flex;
place-items: center;
}
```

This was centering everything, and when the results tried to render, the layout completely collapsed. The app's own styles were fighting with Vite's default stylesheet.

Fix

I replaced index.css with a minimal version:

- {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  }

body {

min-height: 100vh;

background: #030b16;

}

Removed all the default Vite styles that were interfering with the app layout. Finally, the app rendered properly.

---

v0.9 - Groq + Yahoo Finance

Date: March 2026

Status: WORKING

What changed - the architecture is completely different now

This was the breakthrough. Instead of using Claude's API for everything, I split the job between two free services:

- Yahoo Finance (free, no API key) fetches real live data: current price, price change, 52-week high/low, market cap, P/E ratio, EPS, beta, dividend yield, volume.
- Groq API (free tier, 14,400 requests per day) uses the Llama 3.3 70B model to analyze the Yahoo data and generate predictions, technicals, news context, catalysts, analyst consensus, and the full written summary.

Why this works

- Yahoo Finance gives us real live numbers for free, so the prices are always accurate and current.
- Groq is completely free, no per-token cost, no credit card charges, just a free API key.
- The prompt to Groq is small, we just send the Yahoo data (a few hundred tokens) and ask for analysis.
- Cost per analysis: $0.00

Flow of a single analysis

1. User types a ticker (for example AAPL) and clicks Analyse.
2. App fetches live data from Yahoo Finance API, gets real current price, fundamentals.
3. App sends that compact data to Groq with a short prompt asking for full JSON analysis.
4. Groq returns complete analysis: signal, predictions, technicals, news, catalysts, summary.
5. App merges the real Yahoo prices into the Groq response to ensure price accuracy.
6. Results display on screen, full UI with chart, signal ring, news cards, predictions.

Remaining limitations

- News headlines from Groq were still based on training data (not live today's news), but prices were finally real.
- If Yahoo Finance's API was temporarily unavailable, Groq would fall back to using its own knowledge for prices.
- Groq's free tier resets every 24 hours, 14,400 requests is more than enough for personal use.

---

v1.0 - CSS Fix Baked Into App.jsx

Date: March 2026

Status: WORKING

The recurring problem

Even after fixing index.css once, I kept running into the same issue: whenever I shared an updated App.jsx with someone, they would forget to also update index.css. The default Vite index.css has that body display flex and place-items center which collapses the layout and turns the screen black every time.

The permanent fix

I added a CSS override directly inside App.jsx's own style block that fires after index.css and overrides it:

```
html, body, #root {
display: block !important;
place-items: unset !important;
background: #030b16 !important;
}
```

The !important flags override anything in index.css no matter what it contains. Now users only ever need to replace one file, App.jsx, and the black screen cannot come back.

Why this is the right solution

- Single file to update, no more forgetting to also replace index.css.
- Self-contained, the app carries its own layout fix inside itself.

- Permanent index.css can never break the layout again.

---

v1.1 - NewsAPI World Events Added

Date: March 2026

Status: WORKING

What changed

I added NewsAPI as a fourth data source. Now the app fetches real global news, Fed decisions, tariffs, geopolitical events, earnings surprises, and sends it to Groq alongside the Yahoo price data and Finnhub company news.

Full data flow now

- Step 1: Yahoo Finance, fetches live price, P/E, market cap, 52W high/low, EPS, beta, volume.
- Step 2: Finnhub, fetches company-specific news from the past 7 days (earnings, CEO changes, partnerships, lawsuits).
- Step 3: NewsAPI, fetches world events and macro news (Fed rates, tariffs, sector trends, geopolitics).
- Step 4: Groq, receives all three data sources and generates signal, predictions, technicals, catalysts, and full 4-paragraph summary.

Why world events matter

A stock like NVDA can move 10 percent in a day not because of anything the company did, but because of US export restrictions on chips to China, or a Fed rate decision.

Without world event data, the analysis is incomplete. NewsAPI gives Groq the context it needs to make accurate predictions.

Cost breakdown

Service	Purpose	Cost
Yahoo Finance	Live prices	Free, Unlimited
Finnhub	Company news	Free, 60 requests/minute
News API	World events	Free, 100 requests/day
Groq	AI analysis	Free 14,400 requests per day
Total		$0.00 per analysis

---

v1.2 - CORS Fix + startsWith Crash Fix

Date: March 2026

Status: WORKING

Problem 1 - Yahoo Finance blocked by CORS

When the browser tried to call Yahoo Finance directly, it was blocked with: Access to fetch has been blocked by CORS policy: No Access-Control-Allow-Origin header. Browsers enforce CORS rules that prevent calling most external APIs directly from localhost, and Yahoo Finance does not allow direct browser requests.

- Fix: Removed Yahoo Finance entirely. Groq now handles all price data using its own knowledge combined with Finnhub and NewsAPI data.

- Impact: Prices now come from Groq's training data rather than a live feed, but news and world events are still fully live and real.

Problem 2 - startsWith crash on price history

After Groq returned the analysis, the app crashed with: TypeError: p.change.startsWith is not a function. This happened because Groq sometimes returns the priceHistory change field as a number (for example 82) instead of a string (for example "+82%"), and the startsWith method only works on strings.

- Fix: Wrapped the value with String() before calling startsWith, so it works whether Groq returns a number or a string.
- This was causing the black screen - the app received valid data from Groq but crashed when trying to render the 5-year price history chart.

Current data flow - v1.2

- Step 1: Finnhub, fetches company-specific news from the past 7 days.
- Step 2: NewsAPI, fetches world events and macro news.
- Step 3: Groq, receives both news sources and generates full analysis including price data, predictions, technicals, catalysts, and summary.

---

v1.3 - Type Safety Fix - safeStr Helper

Date: March 2026

Status: WORKING

Problem - toLowerCase is not a function

After the CORS fix, Groq started returning certain fields as numbers or null instead of strings. When the app tried to call .toLowerCase() on these values (for MACD, trend, sentiment), it crashed with: TypeError: toLowerCase is not a function.

- MACD field: Groq returned a number like 0.5 instead of a string like "Bullish crossover".
- Trend field: Sometimes returned as null or a number.
- Sentiment field: Sometimes not lowercase, causing news card color to fail.

Fix

I added a safeStr() helper function that converts any value to a string safely before calling string methods on it. Applied it to every field that could be non-string: MACD, trend, sentiment.

```
const safeStr = (v) => v == null ? "" : String(v);
```

This means no matter what Groq returns, the app will never crash on string operations again.

---

v1.4 - safeStr Moved to Top Level Scope

Date: Mid July 2026

Status: WORKING

Problem - safeStr is not defined

The safeStr helper function was defined inside the StockOracle component, but it was being used inside the NewsCard component. In React, child components cannot access

functions defined inside a parent component - they are out of scope. This caused: ReferenceError: safeStr is not defined.

Fix

I moved safeStr to the top level of the file, outside all components, right after the constants. It is now a global helper accessible by every component in the file.

Before: const safeStr defined inside StockOracle on line 276.
After: const safeStr defined at top level on line 14.
No other code changed - just the location of the function.

---

v1.5 - Object Rendering Fix

Date: March 2026

Status: WORKING

Problem - Objects are not valid as a React child

Groq returned analysts.recent as an array of objects like {analyst, target, rating} instead of plain strings. React cannot render objects directly - it only renders strings and numbers. This caused: Error: Objects are not valid as a React child.

Fix

- analysts.recent: Added a check that formats each item as a string if it is an object.

- catalysts.bull and catalysts.bear: Same fix applied - if Groq returns an object instead of a string, convert it safely.
- Root cause: Groq sometimes deviates from the requested JSON structure and returns nested objects instead of flat strings.

---

v1.6 - Prompt Fix - Force Real Prices

Date: March 2026

Status: WORKING

Problem - All prices showing $0.00

Groq was returning zeros for all numeric fields (currentPrice, priceHistory, predictions, technicals). The original prompt asked for real data but did not explicitly forbid zeros or provide reference prices, so Groq defaulted to 0 when uncertain.

Fix - Explicit prompt with reference prices

- Added CRITICAL RULE: "NEVER return 0 for any price field."
- Added hardcoded approximate prices for all 12 tickers so Groq has a reference: NVDA=$115, AAPL=$220, TSLA=$280, MSFT=$420, GOOGL=$175, AMZN=$220, META=$610, SPY=$570, AMD=$110, JPM=$240, BTC-USD=$85000, NFLX=$980.
- Restructured prompt to include a concrete JSON template with REAL_PRICE placeholders so Groq knows exactly what format is expected.

Note on price accuracy

Prices now come from Groq's training data, not a live feed. They are approximate and may be slightly outdated. The news and analysis sections remain fully live via Finnhub and NewsAPI.

---

v1.7 - JSON Repair - Malformed Response Fix

Date: March 2026

Status: WORKING

Problem - JSON parse error at position 1406

Groq occasionally returns slightly malformed JSON - trailing commas, unquoted keys, or truncated arrays. The standard JSON.parse() throws immediately on any error, causing a full crash.

Fix - JSON repair step before parsing

I added a repair pass before JSON.parse that:

- Strips trailing commas in objects and arrays
- Fixes single-quoted values
- Removes control characters
- Adds a fallback: if parsing still fails, the code finds the last valid section and replaces only the summary field with a retry message, so the rest of the data still renders

Now minor Groq formatting glitches no longer crash the app.

---

v1.8 - UI Polish - Ticker Spacing + Prediction Detail

Date: March 2026

Status: WORKING

Fix 1 - Ticker buttons bunched together

Added whiteSpace: nowrap to ticker buttons so they never collapse and always render with proper spacing between them.

Fix 2 - Prediction reasoning too short

Updated the prompt to require 2 to 3 sentence explanations for each prediction period (tomorrow, 7-day, 30-day) instead of a single word. This gives users actual reasoning behind price targets.

---

v1.9 - Scan History + Detailed Analysis

Date: March 2026

Status: WORKING

Feature 1 - Scan History

Added a Recent Scans panel that appears below the ticker buttons on the home screen after the first analysis. Each row shows the ticker, company name, signal, confidence, price, and the time of the scan. Clicking any row re-runs the analysis for that ticker. Stores up to 10 recent scans, deduplicating by ticker so the same stock always appears only once with the latest scan.

Feature 2 - Detailed analysis content

Completely rewrote the Groq prompt to require much more detailed content throughout:

- Summary section: 4 paragraphs of 5 plus sentences each
- News impact fields: 2 to 3 sentences explaining what happened, why it matters, and what to watch for
- Prediction reasoning: 2 to 3 sentences with specific technical and fundamental justification
- Catalyst bullet points: Full sentences of 15 plus words

---

v2.0 - Full Rebuild - Clean File Structure

Date: March 2026

Status: WORKING

Problem - export may only appear at top level

The prompt replacement in v1.9 accidentally inserted the new prompt text before the closing backtick of the template literal. This caused the rest of the file to be interpreted as a second module, pushing export default function StockOracle to line 427 which was inside the try block of the first copy. This caused a fatal parse error.

Fix - Complete file rebuild

I rebuilt the file from scratch by carefully splicing together only the correct sections:

- The helper components from the first copy of the file
- The StockOracle component from the second copy
- Replaced the old prompt with the new detailed Wall Street analyst prompt

All v1.9 features were re-applied cleanly.

Features in v2.0 (all v1.9 features preserved)

- Scan history: Recent Scans panel shows last 10 scans with ticker, signal, price, and time. Click to re-run.
- Detailed analysis: 4 paragraphs of 5 plus sentences each in the full report.
- Detailed news impact: 2 to 3 sentence explanations per news item.
- Detailed prediction reasoning: 2 to 3 sentences per prediction period.
- Detailed catalysts: Full sentences of 15 plus words.

---

v2.1 - All Features Confirmed Working End-to-End

Date: March 2026

Status: WORKING

Status - Full Confirmation

All features confirmed working in live testing with MSFT:

- Real prices ($420.25)
- Live Finnhub news (3 articles with 2 to 3 sentence impact)
- Detailed predictions with reasoning paragraphs
- Technical indicators
- Bull/bear catalysts as full sentences
- Analyst consensus with named firms
- 4-paragraph AI full report
- Scan history all rendering correctly

Fixes applied in v2.0 / v2.1 cycle

- Removed duplicate export default function caused by bad prompt insertion
- Fixed escaped backtick at end of prompt template literal
- Added missing const [scanHistory, setScanHistory] = useState([]) state declaration
- Removed duplicate const [error, setError] line introduced during manual edit

---

v2.2 - Chart Fix + 10 News Items

Date: March 2026

Status: WORKING

Fix 1 - Price history chart not rendering

The MiniChart canvas component was measuring offsetWidth at mount time, but inside a CSS grid the container has 0 width at that moment. The chart drew at 0x0 pixels and showed nothing - only the year labels and percent changes were visible below.

Fix: Replaced the useEffect draw with a ResizeObserver that fires the draw function as soon as the parent container has real dimensions. The canvas now always fills its container correctly regardless of when the component mounts relative to the layout.

Fix 2 - News capped at 3 items

Finnhub and NewsAPI were both fetching up to 8 items but only 3 were being sent to Groq and rendered.

- Fix: Updated to fetch 10 from each source, send 8 of each to Groq, and explicitly instruct Groq to include all provided headlines rather than picking just 3.

---

v2.3 - Twelve Data - Real Live Prices

Date: March 2026

Status: WORKING

The last gap - approximate prices

Until now, all price data came from Groq's training knowledge - approximate values that could be weeks or months out of date. Twelve Data provides real-time stock prices on a generous free tier of 800 requests per day.

What Twelve Data provides

- Current price: real live market price fetched fresh every analysis
- Price change: exact dollar and percent change for the day
- 52-week high and low: real values, not approximations
- 5-year monthly time series: 60 data points used to build the price history chart with real historical prices

How it integrates

1. Step 1 now calls Twelve Data quote endpoint for current price, change, and 52W range
2. Step 1 also calls Twelve Data time series endpoint for 60 months of history, sampled into 6 yearly points for the chart
3. After Groq returns its analysis, the app overwrites Groq's approximate prices with the real Twelve Data values
4. If Twelve Data is unavailable (rate limit, network error), the app falls back silently to Groq's approximate prices - no crash

---

v2.4 - News Limit Fix - 5 Items Only

Date: March 2026

Status: WORKING

Problem - Still showing 10 news items

Even though we were fetching 10 items and sending 8 to Groq, the AI was still generating 10 news items in its response. The prompt was telling it to "include ALL news items provided (up to 10)", so it was doing exactly that.

Fix - Updated prompt instructions

I changed the CRITICAL RULES section from:

"6. news[] = include ALL news items provided (up to 10), not just 3. Use every headline from the company news and world news data above."

To:

"6. news[] = include ONLY the TOP 5 MOST RELEVANT news items from the data provided. Do not include more than 5. Use the actual dates from the news data."

I also updated the JSON template to show exactly 5 items with clean formatting.

Result

The app now shows exactly 5 news items with real dates from the news data. Much cleaner and less cluttered.

---

v2.5 - Dynamic Search Query for Any Ticker

Date: March 2026

Status: WORKING

Problem - Hardcoded "Apple Inc" in search query

The NewsAPI search query was still hardcoded with "Apple Inc" OR "Apple stock", which meant it only worked properly for AAPL. Searching for any other ticker would return irrelevant news.

Fix - Dynamic search query

I replaced the hardcoded search query with a dynamic version that works for any ticker:

```
const searchQuery = '("' + t + '" stock OR earnings OR forecast OR product OR launch)';
```

Now when you search:

- AAPL searches for "AAPL" stock OR earnings OR forecast OR product OR launch
- TSLA searches for "TSLA" stock OR earnings OR forecast OR product OR launch
- BTC-USD searches for "BTC-USD" stock OR earnings OR forecast OR product OR launch

I also updated the app to fetch 10 articles from NewsAPI but only display the top 5 most relevant ones.

Final result

The app now works perfectly for any ticker you search for:

- AAPL shows Apple news
- TSLA shows Tesla news
- MSFT shows Microsoft news
- BTC-USD shows Bitcoin news

All with exactly 5 news items using real dates.

---

v2.6 - StockTwits Sentiment Panel (FAILED)

Date: March 2026

Status: NEVER WORKED

What I attempted

I wanted to add social media sentiment to the analysis - actual conversations from traders and investors discussing the stock in real time. StockTwits is the go-to platform for this, with its bullish and bearish tags on every message. The plan was to fetch the last 100 messages for any ticker, count the bullish versus bearish tags, and display a live sentiment score from the trading community.

Why it failed completely

StockTwits requires a paid enterprise API plan for anything beyond their most basic public endpoints. The credentials I added in v2.6 never worked - not because they were wrong, but because the endpoint I was trying to call simply is not accessible to

individual developers without a commercial agreement. Every request returned a 403 Forbidden or 401 Unauthorized regardless of what I tried.

Annoying things

The StockTwits panel in v2.6 looked great in the UI. It had a clean progress bar, a sentiment percentage, and a little icon showing bulls versus bears. But it never once displayed real data. Every single analysis showed "No data available" because the API call was failing silently. I shipped a feature that looked functional but was completely broken underneath.

---

v2.7 - SEC EDGAR Filings Panel (FAILED)

Date: March 2026

Status: NEVER WORKED

What I attempted

I wanted to pull the most recent SEC filings for each company - 10-K annual reports, 10-Q quarterly reports, 8-K significant event filings. The idea was to show investors when the company last filed and what to watch for. SEC EDGAR is the official source and it is completely free with no API key required. What could go wrong.

Why it failed completely

CORS. The SEC blocks all direct browser requests through their CORS policy. No React app running in a browser can ever call their servers directly without a backend server acting as a middleman. The browser sends a preflight OPTIONS request before the actual GET request, the SEC responds with no CORS headers allowing localhost, and the browser refuses to proceed.

The ugly truth

Just like StockTwits, the SEC EDGAR panel in v2.7 looked polished and professional in the UI. It had a nice table showing filing dates, document types, and clickable links. But it never worked even once. Every analysis showed "No recent filings found" because the browser was silently blocking every request. I spent days on a feature that was fundamentally impossible in a client-side only app.

---

v2.8 - Replaced Both with Three Working Alternatives

Date: March 2026

Status: WORKING - CURRENT VERSION

What I learned

After two versions of shipping features that looked good but never worked, I realized I needed to be brutally honest about what is actually possible in a browser-only React app. StockTwits requires a paid enterprise plan. SEC EDGAR blocks all browser requests. Neither was ever going to work without a backend server - and adding a backend would kill the simplicity of a single-file app that runs anywhere.

The fix - three alternatives that actually work

Instead of pretending, I replaced both panels with three new data sources that are all fully accessible from the browser with real working APIs.

Panel 1 - Fear and Greed Index

Source: alternative.me

Cost: Free, no API key required

CORS: Works perfectly

This index measures overall market psychology on a scale from 0 to 100. When the market is fearful (scores 0 to 25), investors are panicking and selling - often creating buying opportunities. When the market is greedy (scores 75 to 100), investors are euphoric and buying everything - often signaling a top is near.

The API returns a single number once per day, updated every 24 hours. No authentication, no rate limits, no CORS issues. It just works.

What it adds to the analysis:

- Context for whether the stock is moving with or against the broader market
- A reality check: even strong stocks get sold off in extreme fear environments
- A sense of whether sentiment is shifting toward fear or greed over time

How it looks in the UI:

The panel has a red-to-green gradient bar with the current score displayed prominently. The color changes based on the value:

- 0 to 25: Extreme Fear (red)
- 26 to 45: Fear (orange)

- 46 to 55: Neutral (yellow)
- 56 to 75: Greed (light green)
- 76 to 100: Extreme Greed (green)

Panel 2 - Alpha Vantage News Sentiment

Source: Alpha Vantage

Cost: Free tier (5 calls per minute, 500 calls per day)

API Key: [redacted]

CORS: Works perfectly

Alpha Vantage provides AI-scored sentiment on recent news articles for any ticker. It pulls up to 50 articles, scores each one from -1 (very bearish) to +1 (very bullish), and returns both the raw scores and a summarized classification (Bullish, Bearish, or Neutral).

How it works under the hood:

1. App calls NEWS_SENTIMENT endpoint with the ticker
2. Alpha Vantage returns up to 50 recent news items with title, source, publication date, overall sentiment score per article, relevance score for the ticker
3. App averages all the scores into a single number and maps it:
    - above 0.15 to Bullish (displayed in green)
    - below -0.15 to Bearish (displayed in red)
    - in between to Neutral (displayed in yellow)
4. App also shows the raw average score and article count for transparency

Why this matters:

News moves markets. A stock can have perfect technicals and still drop 10 percent on a negative article. Having AI-scored sentiment gives users an early warning system for shifting narratives around the stocks they are watching.

How it looks in the UI:

The panel shows a colored dot matching the sentiment classification, the label (Bullish, Bearish, Neutral), the normalized score out of 100, and a smaller raw score with article count for transparency.

Panel 3 - Finnhub Earnings Calendar

Source: Finnhub

Cost: Free tier (already in app for company news)

API Key: [redacted]

CORS: Works perfectly

This uses the Finnhub key already in the app - the same one powering the company news - to pull the next scheduled earnings date for any ticker. It searches the next 90 days and returns the date, EPS estimate, and revenue estimate if available.

Critical feature - earnings warning:

When earnings are within 14 days, the panel turns red and shows a warning message. This is intentional. Stocks can move 10 to 20 percent in either direction on earnings day. Knowing an earnings date is coming completely changes how you should interpret the analysis - options strategies, position sizing, and risk tolerance all need to adjust.

What the panel displays:

- Next earnings date (formatted clearly)
- EPS estimate if available
- Revenue estimate if available
- Red background warning if within 14 days

Why this is essential:

Earnings dates are the single most important catalyst for most stocks. A stock that looks technically perfect might be a terrible buy two days before earnings because implied volatility is through the roof. This panel makes that risk visible at a glance.

Why this version is actually working

Unlike v2.6 and v2.7, every single panel in v2.8 displays real live data on every analysis:

Panel	Data source	Status	Real Data?
Fear and Greed index	alternative.me	Working	Market mood 1-100
News Sentiment	Alpha Vantage	Working	AI scored article sentiment
Earnings Calendar	Finnhub	Working	Real earnings dates and estimates

The three panels in the UI

Each panel has its own distinct color scheme and icon:

- Fear and Greed: Red, orange, green gradient bar with score and classification. Includes a small explanation of what the index measures for new users.
- News Sentiment: Purple section with Bullish, Bearish, Neutral label and raw score. Shows article count so users know how much data the sentiment is based on.
- Earnings Calendar: Amber section with date, estimates, and urgent red warning when earnings are within 14 days.

What the prompt tells the AI

1. Reference the Fear and Greed score in the analysis - explain what the current market mood means for this specific stock. For example: "With the market in Extreme Fear territory, even strong earnings may not prevent a selloff in the short term."
2. Factor the Alpha Vantage sentiment into the prediction reasoning - if news is bearish, the stock may need more time to recover. If news is bullish, that supports a higher target.
3. Treat earnings dates as critical - if earnings are within 30 days, mention that explicitly and adjust price targets to reflect expected volatility. The prompt says: "If earnings are within 30 days, factor that into the targets - stocks often drift into quiet periods before earnings and can gap hard after."

The updated prompt - key additions

The CRITICAL RULES section now includes:

"2b. Use the MARKET SENTIMENT (Fear and Greed) and NEWS SENTIMENT scores in your analysis - mention whether the market mood is fearful or greedy and what that means for this stock specifically."

"2c. If UPCOMING EARNINGS data is provided, prominently mention the date and what investors should expect. Earnings dates are critical - flag if earnings are within 30 days."

And the summary paragraph instructions now end with:

"Reference the Fear and Greed score and Alpha Vantage news sentiment in context. If earnings are upcoming, factor that into the targets."

Final lesson

v2.6 and v2.7 were embarrassing. I shipped features that looked complete but never worked because I did not verify the APIs were actually callable from a browser. v2.8 fixes that by using only APIs that are:

- Fully documented
- Free tier accessible
- CORS-enabled for browser requests
- Actually returning data on every ticker

The app is now more honest, more useful, and actually works.

Current API Keys and Services (as of v2.8)

Service	Purpose	API Key	Limit
Twelve Data	Real live prices, 52 W range, 5 year price chart history	[redacted]	800 requests/day free
Groq	AI analysis, predictions, signals, summary	[redacted]	14,400 requests/day free
Finnhub	Company-specific live news + earnings calendar	[redacted]	60 requests/minute free
News API	World events, macro news, geopolitics	[redacted]	100 requests/day free
Alpha vantage	AI-scored news sentiment	[redacted]	5 calls/minute, 500 calls/day
alternative.me	Fear and gread index	[redacted]	unlimited

---

v2.9 - Real Data Replaces AI-Generated Numbers + Probability Ranges and Conviction Scoring

Date: March 2026

Status: WORKING - CURRENT VERSION

What changed - The end of AI-generated numbers

The most significant change in version 2.9 is what the AI stopped doing. In every previous version, Groq was responsible for generating the technical indicators, analyst consensus numbers, price targets, P/E ratios, EPS, and beta. Those numbers looked convincing but were essentially educated guesses - the model was drawing on training data that could be months out of date and had no mechanism to verify any of it. Version 2.9 fixes this by fetching all of those numbers from real APIs and handing them to Groq as facts to interpret rather than figures to invent.

Real technicals from real price data: RSI, MACD, MA50, and MA200 are now calculated in JavaScript directly from the five years of monthly price data already being fetched from Twelve Data. The formula is standard - log returns, exponential moving averages, the full RSI calculation with average gains and losses over 14 periods. The AI receives the output and is explicitly told not to change or recalculate any of it.

Expanded Finnhub integration - five new endpoints: Previously Finnhub was only used for company news. Version 2.9 adds:

1. Analyst recommendation trends: Real buy, hold, and sell counts from actual analysts. The panel now shows a proportional breakdown bar so you can see at a glance whether consensus is heavily bullish or split.
2. Price targets: Real mean, high, and low price targets with last updated date.
3. Fundamental metrics: P/E, EPS, beta, revenue growth, and gross margin pulled directly from Finnhub.
4. Insider transactions: Real dollar amounts of executive buying and selling over the last 90 days, grouped by person and showing net buys or sells.
5. Earnings surprises: Actual versus estimated EPS for the last four quarters, showing whether the company beat or missed.
6. Company profile: The sector, industry, and official company name now come from Finnhub rather than the AI's memory.

SEC filings now work: They are now routed through Finnhub instead of querying the SEC directly, which had been blocked by CORS since version 2.7. It finally works.

v2.9 - Probability Ranges and Conviction Scoring

The prediction panel was replaced entirely. The old version showed a specific price target like "+2.1% in 7 days" which was a single number invented by the AI with no statistical basis. The new panel calculates real historical volatility from the price history using log returns and the square root of time rule - the same method used in professional options pricing.

Instead of one invented number, the app now shows:

- A 68 percent probability range (1 standard deviation)
- A 90 percent probability range (1.645 standard deviations)

for tomorrow, 7 days, 30 days, and one year. The 68 percent range means that based on how this stock has actually behaved historically, the price has landed inside that band 68 percent of the time over equivalent periods. That is a mathematically honest statement rather than a false precision.

The conviction scoring panel was added as a way of communicating when multiple proven signals align. Six signals are evaluated on every analysis:

1. RSI mean reversion (oversold equals bullish about 57 percent edge, overbought equals bearish about 55 percent edge)
2. Price relative to the 200-day moving average (trend following about 54 percent edge)
3. MACD direction (53 percent edge)
4. Net insider buying or selling (up to 60 percent edge)
5. Earnings surprise momentum (post-earnings drift about 55 to 60 percent edge)
6. Analyst consensus strength (upgrade momentum about 56 percent edge)

Each signal has a historical edge percentage sourced from academic finance research. When four or five signals point in the same direction the app shows STRONG BULL or STRONG BEAR, meaning the historical probability of that direction meaningfully exceeds a coin flip. When signals are mixed it shows NEUTRAL and makes no strong directional claim. A disclaimer is shown below the panel making clear these are statistical averages from research, not guarantees, and that markets can override any model at any time.

Why this version is the most honest yet

- Technicals are now real - calculated from actual price history, not invented by an LLM.
- Analyst ratings and fundamentals are real - pulled from Finnhub, not guessed.
- Price targets are real - from actual analyst reports via Finnhub.
- Volatility ranges are statistically valid - based on historical log returns.
- Conviction scoring is transparent - you see exactly which signals are aligned and what their historical edge is.

The AI's job is now to interpret real data and explain it in plain English, not to make up numbers that look convincing. This is a fundamentally more honest application of AI to financial analysis.

Stock Oracle v3.0 — Node.js Backend + SEC EDGAR Proxy + Full Finnhub Integration

Date: March 2026

Status: Working (Current Version)

Overview

Version 3.0 introduces the most important upgrade to Stock Oracle so far: a Node.js backend.

Earlier versions (2.6 and 2.7) revealed a major limitation of browser-only apps. Some services simply cannot be accessed directly from the frontend. For example:

- SEC EDGAR blocks browser requests because of CORS restrictions.
- StockTwits requires an enterprise plan for API access.

No amount of frontend code could solve this.

Version 3.0 fixes the problem by adding a backend server that acts as a proxy. The backend makes requests to restricted APIs and then sends the data to the frontend in clean JSON.

The backend runs on **http://localhost:3001**, while the frontend runs on **http://localhost:5173**.

Backend Architecture

The Node.js backend handles requests that browsers cannot make directly.

Example endpoint:

```
app.get('/api/sec/filings/:ticker', async (req, res) => {

  const { ticker } = req.params;



  // 1. Convert ticker to CIK

  // 2. Fetch filings from SEC EDGAR

  // 3. Return clean JSON to the frontend

});
```

This proxy approach allows the frontend to safely access restricted APIs without running into CORS blocks

Additional data sources:

- **Twelve Data** – live prices and historical charts
- **Alpha Vantage** – news sentiment scoring
- **NewsAPI** – global events and macro news
- **alternative.me** – Fear & Greed Index

Features Now Working

Stock Oracle v3.0 currently includes:

- Live prices and historical charts
- Real technical indicators calculated from price history
- Analyst ratings and price targets
- Company fundamentals and metrics
- Company news and world news
- AI-scored sentiment analysis
- Fear & Greed market psychology indicator
- Earnings calendar with warnings
- Insider trading data
- Earnings surprise tracking
- SEC filings via backend proxy
- Probability ranges based on historical volatility
- Conviction scoring across signals
- AI analysis using Groq
- Scan history tracking
- Exactly five news items per analysis
- Works for any ticker

All analyses run **without cost per query**.

---

v3.1 — Deployment Fixes and Public Release

Date: March 11, 2026

Status: WORKING — PUBLIC (Code - except the website link and the API)

What changed

Fixed environment variable naming in Render (corrected GROQ_KEYS to GROQ_KEY).

Updated frontend Groq API call to use backend proxy instead of direct API call with key.

Verified all environment variables are correctly set and accessible.

Made code repositories public on GitHub for transparency and collaboration.

Successfully deployed and tested live application at https://stock-oracle-v3.netlify.app with all features working.

Why this matters

This version represents the first fully functional public release of Stock Oracle. Users can now access live stock data, AI analysis, SEC filings, and all other features without any setup. The backend proxy architecture ensures API keys remain secure while allowing full functionality.

Current status

All features confirmed working:

- Live prices from Twelve Data
- Technical indicators calculated from real price history
- Analyst ratings and fundamentals from Finnhub
- News and sentiment from NewsAPI and Alpha Vantage
- SEC filings via backend proxy

- AI-powered analysis via Groq
- Probability ranges and conviction scoring
- Works for any ticker

Cost per analysis: $0.00 (all APIs free tier)

How to Run Stock Oracle

If you are reading this and want to run the app yourself, follow these steps:

1. Open Command Prompt
2. Navigate to the project folder.
   cd C:\Users\Dominique\stock-oracle
3. Start the development server.
   npm run dev
4. Click the link that appears (usually http://localhost:5173) while holding down the Control key
5. Start the backend server: cd C:\Users\Dominique\stock-oracle-backend → node server.js

That is locally the website is: [redacted]

The app will open in your browser. Type any ticker (AAPL, TSLA, MSFT, BTC-USD, and so on) and click ANALYSE to see the full AI-powered analysis with real-time data from all six sources.